

University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Jakub Bednarz

Student no. 406103

Investigating and Combining Approaches for Data-Efficient Model-based RL

Master's thesis
in MACHINE LEARNING

Supervisor:
dr. Jacek Cyranka
Instytut Informatyki

Warsaw, Nov 2024

Abstract

[Abstract]

Keywords

...

Thesis domain (Socrates-Erasmus subject area codes)

11.4 Artificial Intelligence

Subject classification

...

Tytuł pracy w języku polskim

Analiza i połączenie metod w wydajnym obliczeniowo uczeniu ze wzmocnieniem bazowanym
na modelu

Contents

1. Introduction	5
2. Background	7
2.1. Reinforcement Learning	7
2.2. Deep RL	7
2.3. Benchmarks for sample-efficient RL	8
2.4. Data augmentation in RL	8
2.5. Self-supervised RL	8
2.6. Model-based RL	9
2.6.1. Learning the model	9
2.6.2. Using the model	9
2.7. Training recipes for data-efficient RL	10
2.8. Integrated architectures	10
3. Method	11
3.1. Overall design	11
3.2. Implementation	12
3.2.1. Configuration	12
3.2.2. Environments	13
3.2.3. World model and RL modules	13
3.2.4. Data loaders and model-free mode	16
3.2.5. Training recipes	16
4. Experiments	19
4.1. Experimental setup	19
4.2. Training recipe optimization	21
4.2.1. Base architecture	21
4.3. Naïve speedup test	22
4.3.1. Decoupling model and RL optimization	23
4.4. Investigating the learning dynamics of the agents	23
4.4.1. Analyzing the world model	24
4.4.2. Plasticity analysis	27
4.4.3. Analyzing the RL component	27
4.5. Auto-tuning update frequencies	28
4.5.1. Decoupled adaptive ratio system	29
4.6. Data augmentation	29
4.7. Choice of the RL algorithm choice	34
4.7.1. PPO Experiments	34

4.7.2. SAC Experiments	35
4.8. Combining the improvements	36
4.8.1. Hyperparameter selection	39
5. Discussion	41

Chapter 1

Introduction

Reinforcement Learning (RL), a research field dedicated to developing intelligent autonomous agents, capable of learning to do tasks in various environments, has achieved remarkable progress in recent times. Through the use of deep learning and artificial neural networks, algorithms playing video games at a super-human level or controlling robots without any prior knowledge have been developed.

However, most of the current state-of-the-art techniques have a significant drawback: they require excessive amounts of data (which, in this setting, we regard as the total time spent interacting with the environment) in order to achieve such performance. For example, standard benchmark on a collection of Atari video games assumes a total “budget” of 200 million game frames, equivalent to a human playing for ≈ 40 days without rest. This shortcoming greatly limits real-world deployment of such methods, as without a simulator of the target environment learning the proper behavior is impossible.

In order to remedy this problem, many approaches designed with *sample efficiency*, being the performance obtained with a given (small) number of interactions with the environment, in mind, have been proposed, such as:

- Improving sample efficiency of existing algorithms, without any major architectural changes, by carefully tuning the number of optimization steps per single environment step.
- Learning a model of the environment, with which one can either simulate environment interactions for the model-free method, or use the model in a more sophisticated way, e.g. through a planning algorithm.
- Better architectures of the neural networks comprising the agents.

All these have resulted in marked improvements in terms of sample efficiency. One may observe, however, that such methods have been developed independent of each other, and in many cases are orthogonal, meaning it is possible to envision a single RL agent combining many such strategies and modifications, hopefully capable of reaching even higher performance than standalone algorithms.

With this in mind, the goal of this thesis is to answer following questions:

1. *What are the existing methods for improving data efficiency of RL agents?*
2. *How to approach the design of a single algorithm combining these methods?*

3. *What are the synergies and interferences between these modifications? And, relatedly, what (if any) improvement in terms of performance can we expect from such an agent?*

The rest of this thesis is structured as follows: (...)

Chapter 2

Background

2.1. Reinforcement Learning

Reinforcement Learning is a field of artificial intelligence and machine learning concerned with constructing intelligent agents capable of learning what actions to take in an environment so as to maximize a scalar reward signal.

Formally, the environment can be represented by a Markov decision chain (MDP) in the form of a tuple $\langle \mathcal{S}, \mathcal{A}, p \rangle$ consisting of the state space $\mathcal{S}$, action space $\mathcal{A}$ and transition probability $p(s_{t+1}, r_t \mid s_t, a_t)$, where $r_t \in \mathbb{R}$ denotes the reward granted at that step. Additionally, in the case of partially observable environments, we have an observation space $\mathcal{O}$ and an observation probability $p(o_t \mid s_t)$. The agent, starting from an initial observation o_1 , acts according to a probability distribution $\pi(a_t \mid o_{\leq t})$ conditioned on previous observations until it reaches a *terminal* state s_T . The goal of reinforcement learning and the RL agent is to maximize the expected sum of rewards (so called returns) obtained along the trajectory $G = \mathbb{E} \left\{ \sum_{t=1}^{T-1} r_t \right\}$.

There are two further quantities of interest. The state value function $V^\pi(s)$ equal to the expected returns of an agent acting with a policy $\pi(a_t \mid s_t)$, when starting from a state s . The state-action value function $Q^\pi(s, a)$ (called thereafter Q function) equal to the expected returns of an agent acting with a policy π , when starting from a state s and having just performed action a . These value functions are usually γ -discounted: rather than using the returns $\mathbb{E} \left\{ \sum_{t=1}^{T-1} r_t \right\}$, we consider $\mathbb{E} \left\{ \sum_{t=1}^{T-1} \gamma^{t-1} r_t \right\}$ for $0 < \gamma < 1$. This allows us to use fixed-point arguments in the proofs of various RL theorems. It is also the standard approach taken by prominent RL algorithms.

One approach towards learning behavior in an environment is *Q-learning*. The idea is as follows: we want to learn an optimal policy π^* such, that V^{π^*} is maximized for all states s . It is clear, that such a policy should choose a , which maximizes $Q^{\pi^*}(s, a)$, i.e. π^* should be deterministic, and return $\pi^*(s) = \arg \max_a Q^{\pi^*}(s, a)$. It can be shown, that such a policy, and such an induced Q -function $Q^{\pi^*} =: Q^*$ satisfy $Q^*(s, a) = \mathbb{E}_{s'} \{ r + \gamma Q^*(s', \pi^*(s')) \}$. This recurrence relation allows us, in turn, to learn the value of Q^* through fixed-point iteration.

2.2. Deep RL

[A section about DQN, SAC, PPO and other such stuff?]

2.3. Benchmarks for sample-efficient RL

A number of setups for evaluating data-efficient agents have been established in prior work.

Atari-100k For discrete control, the *de facto* standard benchmark is Atari-100k [2], a subset of 26 games from the full Atari Learning Environment (ref the paper) benchmark, limited to 400×10^3 game frames, equivalent to 100×10^3 agent stops assuming the standard action repeat value of 4.

2.4. Data augmentation in RL

Drawing from the common practice of using data augmentation (e.g. image transformations like rotation, random shifts) in supervised learning to improve performance and reduce overfitting, DrQ develops an RL-specific augmentation schema: beyond simply applying random transforms to observations, we may also use the fact that Q and V functions should be invariant w.r.t. the transforms to smooth them out by averaging.

2.5. Self-supervised RL

Standard deep RL methods, such as DQN or SAC, only utilize the reward signal in order to learn useful state representations to be used to learn the policy. Some authors have proposed employing techniques from the area of self-supervised/unsupervised learning in order to combat this inefficiency.

Broadly speaking, self-supervised learning consists of adding additional *pretext tasks*, which are solved not for their own sake, but rather to provide learning signal for the downstream representations. Many kinds of pretext tasks have been suggested:

- For visual input, one option is to exploit inherent structure of images, via rotation prediction, colorization (reconstructing the image from a grayscale version) or jigsaw (splitting an image into patches, randomizing the order and reconstructing the original image).
- Contrastive learning (CL) approaches, based on the instance discrimination task, construct such representations, that views (augmentations, crops etc.) of the same instance (“positive pairs”) are pulled together, and views of different instances (“negative pairs”) are pulled apart. Some CL methods do not use negative pairs altogether, relying on momentum, batch normalization and projection heads to avoid collapse of the representations.
- Generative approaches reconstruct input from noisy or corrupted versions. Autoencoders, variational autoencoders (VAEs) and denoising VAEs pass the (possibly augmented) input through a latent bottleneck in order to recover the original input. The corruption can also take form of masking patches of the input - this approach has proven especially suited to pre-training representations for NLP and Vision Transformers.

Similarly, self-supervised RL techniques attempt to incorporate pretext tasks into regular RL pipelines. Arguably the simplest way is to apply SSL to augment learning of observation embeddings. For example, CURL takes a base RL model (in this instance SAC) and uses MoCo to help train the observation encoder via an auxiliary CL loss term. Some architectures,

however, design the pretext task to be more RL-specific. SPR, for instance, treats a future state and a state predicted by a transition model from the past state and the sequence of actions in between as a positive pair. Masked World Models use the paradigm of masking image patches and reconstructing them, with an important addition of also predicting rewards, which proves crucial for achieving high final performance.

2.6. Model-based RL

Whereas a model-free RL agent seeks only to learn the policy π , a model-based agent seeks also to model the probability, or simply allow sampling, of future states and rewards $\widetilde{s_{t+1}}, \widetilde{r_{t+1}}, \dots, \widetilde{s_{t+H}}, \widetilde{r_{t+H}}$ given past observations $o_{\leq T}$ and actions to perform $a_t, \dots, a_{t+H-1}$, where H denotes the imagination horizon.

2.6.1. Learning the model

SimPLE performs the transition in input (image) space - the UNet-like model receives input consisting of last K frames, as well as action conditioning, and is trained to output next frame. Stochasticity of the environment is modeled by encoding the input-output pair into a discrete latent vector (intuitively representing which of the possible futures to expect) which is fed to the decoder; the distribution of these latent vectors is modeled autoregressively with an LSTM for use at inference time. The issue with the approach is that encoding and decoding images in such a fashion proves to be computationally expensive. Thus, other model often opt to perform modelling in the latent space. World Models, for example, uses a two-stage training recipe: visual inputs are first compressed into a latent vector using a Variational Autoencoder, whereafter a combination of an RNN and a “distribution head” model the transition probabilities in the latent space. In (PlaNet), the authors raise the issues surrounding modelling stochastic environments autoregressively and propose the use of both deterministic and stochastic recurrent networks in order to deal with stochasticity without inducing the decay of the information stored in the state vector. This approach has been further refined in the Dreamer series of models by e.g. using discrete latents or employing better techniques for pulling together prior and posterior state distributions.

Model states do not necessarily need to correspond to real states - in the case of *value-equivalent models*, the states and the transition function are optimized to approximate the values to be obtained after executing a sequence of actions. (Speak of MuZero &c?)

2.6.2. Using the model

Let us take a look now at how such a world model can be used in an RL agent. One class of methods uses the model for *planning*; such techniques are also known as *lookahead methods*. Here, planning refers to any decision process which actively uses the world model. For example, in its simplest incarnation, Model-Predictive Control (MPC), at every time step, optimizes a sequence of actions $a_t, \dots, a_{t+H-1}$ such that the expected total rewards $\mathbb{E} \left\{ \sum_{k=0}^{H-1} r_{t+k} \right\}$ is maximized, and selects the first action a_t - at the next step, the procedure is performed again. The action sequence can be optimized using any zero-order method, like Monte Carlo (i.e. sampling the actions randomly and selecting the one with the best score), or through Cross-Entropy Method (CEM). A variation of this idea is to employ a Monte Carlo Search Tree in order to focus on more promising directions of exploration. In all cases, an auxiliary value function predictor can also be used in order to approximate the returns beyond the rollout horizon.

At the other end of the spectrum lay approaches which use model rollouts for learning the policy. Such rollouts can be used either as a data augmentation technique, or completely replace training on real data, instead learning behaviors entirely in the “imagination” via regular model-free RL algorithms.

2.7. Training recipes for data-efficient RL

A recent line of research has focused on achieving greater sample efficiency without any architectural changes - rather, the idea is to optimize the training recipe, which we define as the way optimization and environment interaction steps are organized throughout the training process. In some cases, as shown in (Data-Efficient Rainbow), it is sufficient to simply increase the frequency of optimization steps per environment step. At sufficiently high replay ratios, the final performance of the agents starts to decrease. In (Primacy Bias paper), the authors analyze this phenomenon, and suggest that the fundamental cause of this is *primacy bias*, whereby the neural networks overfit to early interactions, causing the loss of ability to learn and generalize from more recent data. The proposed remedy is to periodically fully or partially reinitialize the networks used by the agent - this simple procedure is shown to significantly improve performance in low-sample regime.

2.8. Integrated architectures

The main idea of this paper is inspired and motivated by Rainbow architecture and associated paper. In it, the authors took a base off-policy model-free RL algorithm, DQN, applied an array of modifications developed by other authors (specifically: double Q-learning, dueling networks, prioritized sampling, noisy networks, distributional RL and multi-step TD targets), adapted to fit together in a unified architecture, and achieved substantial improvements in terms of final performance of the agent.

Our goal is to develop a framework for sample-efficient RL, in a similar fashion to Rainbow - taking a base architecture, an array of approaches developed to improve sample efficiency, and combine them together into a single architecture. To our knowledge, this has not yet been investigated.

Chapter 3

Method

3.1. Overall design

Let us now consider how these advancements can be integrated into a single unified, modular framework for constructing sample-efficient RL agents.

The use of world models in a number of state-of-the-art approaches for data-efficient RL informs our choice to design the architecture in a model-based fashion. Thus, the framework shall consist of a world model, the RL agent proper, along with a replay buffer.

In many cases, the advances in sample efficiency result from the development of better model or RL algorithm architectures *in isolation*. With this in mind, we wish to make the framework allow for possibly seamless swapping out of the model and RL modules. In order to do this, we need to examine how different model-based architectures structure the interaction between the model and the RL modules.

1. Dreamer series takes the approach of training a (possibly arbitrary) model-free RL algorithm purely with simulated experience. The model and the RL modules are essentially independent of each other.
2. One may mix real and simulated data for the purposes of training the RL agent. For example, in MBPO, the authors use an on-policy method, but augment the data with imaginary rollouts.
3. The agent module may not even require any training per se. For example, PETS and PlaNet use cross-entropy method (CEM) for selecting actions to perform by planning with the learned model.
4. Methods using value-equivalent models, such as MuZero or EfficientZero, train the model and the RL module jointly. In this instance, the interaction is more complex.

Our desiderata are best fit with a generic approach inspired by the Dreamer series. To be precise, we consider a following scheme:

1. The world model is independent of the RL module, and is optimized on samples from a replay buffer.
2. The RL module may use the model actively, and may require optimization using batches of on- or off-policy rollouts, either real or simulated, or a mixture of both (if the observation and model state spaces coincide.)

This formulation subsumes most of the algorithms we have considered so far, such as Dreamer, MBPO, PlaNet, SimPLe, or IRIS: the specific method can be replicated by adjusting the architectures of the model and of the RL module, or the form of the input data for the RL module optimization step. Data augmentation may, for example, be implemented by placing a layer between the replay data loader and the model.

Beyond the network architecture, a major component of the design of sample-efficient agents is the structure of the training loop, meaning the order and frequency in which environment interaction, model optimization and RL module optimization occur. Indeed, works such as Data-Efficient Rainbow or SR-SPR prove that ordinarily inefficient methods can be made vastly more data-efficient by refining the training process. Thus, it is necessary for the framework to accomodate specifying different kinds of training recipes.

3.2. Implementation

In order to facilitate the experiments, we have implemented a modular model-based RL framework in Python, using the PyTorch library. The source code is accessible at <https://github.com/vitreusx/rsrch>, and is structured as a regular Python package, making it installable.

The entry point for the framework is the `Runner` class, responsible for instantiating the requisite neural networks, RL environments, replay buffers, data loaders, optimizers etc. We will elaborate on the further aspects of the implementation by following a normal flow of the program.

3.2.1. Configuration

First, we must load the configuration. Due to the variety of the experiments and ablations, that the system needs to accomodate, the framework needs to be highly configurable. Our implementation allows for the control of, among other things, *types* of the networks to be used - one may, for example, use IMPALA observation encoder rather than the original one - as opposed to only e.g. the layer sizes; and the entire structure of the training loop. We have found following properties to be crucial throughout the development process:

1. There needs to be a single YAML file with default parameter values.
2. We need to be able to compose *preset files*. These files are, in effect, overwrites of the default config. These presets also need to be extensible. For example, we may create a preset corresponding to a single experiment, containing the parameter values to be overridden, and want to create a variant of that experiment by changing a single value. We may also want to combine multiple presets, for example in order to copy environment specification from one preset, and training details from another.
3. The solution must allow for variable interpolation, meaning the capacity for one parameter to reference other values in the YAML file. The interpolation cannot be limited to strings alone. Also, we would like to be able to perform mathematical operations to compute certain parameter values on-the-fly. The interpolation ought to be performed after all the presets have been merged into the base configuration object.
4. The raw configuration object (a dictionary) needs to be convertible to a Python class, preferably a `dataclass`, in order to ensure type safety and increase the ease of development via the presence of the type hints.

Of the existing solutions, we have found `omegaconf` to be the one best matching our desiderata.

3.2.2. Environments

After parsing the configuration file, we must set up the RL environments, such as the Atari games, or DMC tasks. Our main design goal concerning the construction and use of the RL environments has been to separate the implementation details of the environments, and the rest of the project.

The main issue is as follows: the components further down the line, such as the actor networks or the world model, accept the PyTorch tensors as the input, whereas most RL libraries return Numpy arrays. The replay buffer(s) should also store Numpy arrays. We may note, that in some common cases, none of these “data formats” are not the same, even the replay buffer data and the data obtained from the environment. For example, many Atari agents utilize *frame stacking*, whereby last K emulator frames are passed to the agent. Clearly, we would like to store only the most recent one in the buffer, to avoid wasting memory - in fact, for a replay buffer of size 2×10^6 and grayscale observations of size 86×86 , we would need $\approx 60\text{G}$ of memory to store all of them in an uncompressed form if not doing any memory optimization. Beyond this, we need to make sure, that the data transforms of the environment samples and buffer samples yield the same results. Usually, all these nuances are handled manually, but this approach introduces a lot of possibilities for inadvertently adding hard-to-fix bugs.

Ideally, then, we would like to hide all of this from the end user. The way we achieve this is through an *environment SDK*. The SDK’s role is to expose a minimal interface for creating environments, performing interactions with them, and creating replay buffers to store the experience. The Python-esque pseudocode is provided in Listing 1. The idea is to provide an unified interface for acting in the environment(s) via a `VecAgent` protocol, and to hide the details concerning various conversions, data storage details etc. through `wrap_buffer` and `rollout` methods of the SDK.

Another crucial feature is the presence of `obs_space` and `act_space` member fields. They provide information concerning the format of the data received by the agent or sampled from the buffer. We provide a set of classes mirroring the `gym.spaces` classes, such as `Image` or `Box`, but for the PyTorch tensors, instead of Numpy arrays.

We provide SDKs for Atari Learning Environment (ALE), DeepMind Control Suite (DMC) and a generic one for `gymnasium` environments. For ALE and DMC, we use optimized, C++-based implementations provided by `envpool` library [Add the paper citation]. We may note that, despite `envpool` not sharing API with Gym environments, we do not need to make any changes to the rest of the code, like the training loop, showcasing one major advantage of using such SDKs over handling all the environment-related code manually.

3.2.3. World model and RL modules

In order to implement swapping out the world model and the RL algorithm implementations, we separate them out into replaceable modules with unified interfaces.

As regards the world model interface, it is difficult to devise a single common scheme for all possible methods we may envision. Some algorithms, for example, perform transitions in observation space, whereas others operate in a latent space. The model can either give the probability distribution over future states, or act in an implicit manner, e.g. by generating future states from noise via Diffusion Models. Certain approaches model the environment

Listing 1 Pseudocode for environment SDKs

```
class VecAgent:
    def reset(self, indices, observations):
        """Receive first observations of the new episodes in the
        environments specified by the indices."""

    def policy(self, indices):
        """Choose actions to perform in given environments, as specified by
        the indices."""

    def step(self, indices, actions, observations):
        """Observe new observations and actions leading to them in the
        environments specified by the indices."""

class SDK:
    obs_space: Any
    """Observation space, in tensor format."""

    act_space: Any
    """Action space, in tensor format."""

    def make_envs(self, num_envs: int, **kwargs):
        """Create a number of environments. The implementation can be
        optimized, e.g. by vectorizing the environments."""

    def wrap_buffer(self, buffer):
        """Adapt a regular replay buffer into an SDK-aware version.
        When adding data obtained from the environment, it is converted to
        the buffer format. When sampling the data, the samples are converted
        to the tensor format."""

    def rollout(self, envs, agent: VecAgent):
        """Create a stream of interactions of an agent with the environment(s).
        The agent receives data in the tensor format, and should output actions
        in the tensor format as well."""
```

only in the sense, that they allow reconstruction of the rewards, and the “states” produced do not correspond to the actual environment states.

We take the path of requiring the model to implement (1) an inference process $s_{t+1} \sim p(s_{t+1} | s_t, a_t, o_{t+1})$, (2) performing an imaginary rollout $\tilde{s}_{t+1:t+H}, \tilde{r}_{t+1:t+H} \sim p(\tilde{s}_{t+1:t+H}, \tilde{r}_{t+1:t+H} | s_t, a_{t:t+H-1})$, where H is the horizon length. This choice excludes the use of RL algorithms which explicitly require access to the transition probability distribution, but in practice none of the algorithms considered so far had such a requirement.

In the case of RL algorithm modules, we require the sampling of the next action, given the current state $a_t \sim \pi(a_t | s_t)$ - the actor is, in effect, memoryless, and we will rely on the world model to encode all the information necessary to perform optimal actions.

During the training process, additional code and networks are necessary to perform optimization. We split these responsibilities off to a separate **Trainer** class. For example, a **Trainer** for a world model may contain, aside from the optimizers, observation decoder, which is strictly speaking not necessary during inference. Similarly, a trainer for an actor-critic RL algorithm would contain the value and Q functions, as well as the target versions (if using double Q learning).

We provide implementations of DreamerV2’s world model and RL algorithm - a variant of Advantage Actor-Critic. We follow the original source code for the agent `danijar/dreamerv2`. Let us note, that the authors’ reference implementation has been written for the Tensorflow deep learning framework. We have considered a number of open-source reimplementations of Dreamer in Pytorch, and found them to be inadequate in some ways:

1. `jsikyoondreamer-torch` is a very close translation of the original codebase, suffering from the same issues, mainly the code quality.
2. `jurgisp/pydreamer` contains a number of features not present in the original implementation, such as variable model update ratios, using an auxiliary actor-critic in the model, and importance-weighted advantage estimation.
3. `pytorch/rl` and `juliusfrost/dreamer-pytorch` only implement DreamerV1.
4. `Eclectic-Sheep/sheeprl` is a distributed RL framework.

As none of the considered options proved either usable as a Python package, or clear enough to “copy-paste” into the framework in the form of a standalone module, as described above, we have decided it would be better to reimplement it ourselves. Beyond merely replicating the Tensorflow code, we have added the ability to specify and change the encoder and decoder architectures for observations, rewards and termination signals. During development process, we have also found, that the eager-evaluation nature of Pytorch hurts the performance, compared to Tensorflow implementation, which uses JIT. Thus, we have also implemented a `torch.jit.script` version of the Recurrent State Space Module in PyTorch, which has increased the number of iterations per second by 25%.

Also, we provide implementations of two model-free algorithms: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). We base our implementations on the ones found in `vwoxyzjn/cleanrl`, with a number of changes made to make them usable in the model-based context:

1. The implementation of SAC used a one-step TD target $y = r + \gamma V(s')$. In order to bring it in line with PPO and A2C, we change it to Generalized Advantage Estimation (GAE).

2. Since the “observations” given to the model are in fact world model states, we change the default convolutional encoders to the feedforward networks used in the A2C implementation.
3. Imaginary sequences generated by the model are unlike the regular sequences drawn from e.g. the replay buffer. Because we do not know *a priori* whether a sampled state is terminal or not - we may only use the terminality predictor of the model, which would return the probability that a state is terminal or not - we need to account for a degree of “fuzziness” (in the sense of fuzzy logic) of the latent states. The alternative could be to stop a particular rollout during the generation process, when the termination probability exceeds a given threshold. In that instance, however, we would have to deal with batches of sequences of non-uniform length, which would likewise require changes to the implementation of the RL algorithm, and would likely be computationally inefficient, since we could not perform batched operations easily. We opt, therefore, for the former approach; this is also the approach taken in the DreamerV2’s implementation of A2C. In practice, this nuance requires two implementation changes to be made: (1) per-item losses need to be multiplied by a weight factor, to prevent unreal states from affecting the optimization process, (2) GAE and other value estimation methods need to be changed in order to accept non-constant γ discount factors.

3.2.4. Data loaders and model-free mode

Given the replay buffer, the next step is to sample sequences from it, and collate them into a batch to be used for model or RL optimization. We extract this process into a separate *data loader* class, similar to PyTorch `torch.utils.data.DataLoader`. In this way, we make the training loop more readable, and more similar to the regular training loops used for the supervised tasks such as image classification or next-token prediction. The other advantage is the ability to experiment with different kinds of data loaders. We may, for example, use a variant with prioritized sampling, or with data augmentation added.

We can also use this functionality to implement a model-free debug mode. The idea is as follows: when we add a new RL algorithm to test, it would be wise to first test it in a model-free setting, to assess whether any possible agent performance issues are caused by the RL algorithm. This can be achieved in a following fashion:

1. Ordinarily, the RL data loader loads a batch of real-world sequences, performs inference process to obtain the state for each batch item and time step, and performs a rollout “in imagination” using the behavior policy and the model. We can, however, replace it by a regular data loader, loading batches of real-environment sequences.
2. There remains adjusting the various encoders and other networks to accept environment samples (e.g. images), as opposed to model states. This can be done simply through making changes to the configuration of the RL module.

3.2.5. Training recipes

After the stage of setting up neural networks, replay buffers, environments etc. we proceed to the training process proper. The main issue is the vastitude of the possible recipes - we could load and/or save checkpoints, load external data to replay buffer, want to use different schedules of performing model and RL training steps, perform validation epochs throughout the process and at the very end, reset network parameters at different time points etc. In

Listing 2 An example `stages` config field.

```
stages:
- prefill:
  until: 20e3
- train_loop:
  until: 400e3
  tasks:
    - do_val_epoch: ~
      every: 20e3
    - do_wm_val_step: ~
      every: { n: 16, of: wm_opt_step }
    - do_rl_val_step: ~
      every: { n: 16, of: rl_opt_step }
    - do_wm_opt_step: ~
      every: 8
    - do_rl_opt_step: ~
      every: 4
    - do_env_step
- do_val_epoch
- save_ckpt:
  full: false
  tag: final
```

order to resolve this issue in as straightforward and user-friendly a manner as possible, we have elected to allow the user to specify the recipe manually in the configuration file.

The relevant configuration field is `stages` - an example can be found in Listing 2. The task names (`prefill`, `do_wm_opt_step` etc.) correspond to functions of the `Runner` class. We specify the frequency or duration of various actions through `until` and `every` fields - for example, we can see that buffer prefill phase lasts until $P = 20 \times 10^3$ environment steps have passed. The training loop lasts until $T = 400 \times 10^3$ environment steps, and consists of validation epoch every 20×10^3 steps, model optimization step every 8 environment steps, RL optimization step every 4 environment steps etc. Finally, we do a final validation epoch and save the checkpoint.

Such an example experiment configuration would have been difficult to specify precisely, if we were to use pre-defined experiment presets, and would be far less comprehensible.

Chapter 4

Experiments

4.1. Experimental setup

We are interested in evaluating performance on the Atari-100k benchmark. However, performing training runs on the entire set of 26 games and with a full range of RNG seeds for every modification is computationally infeasible. Thus, we will use a subset of these games and a reduced number of seeds as a proxy. To be specific, we shall limit ourselves to a choice of 3 environments and 5 RNG seeds. In order to properly gauge the effect of changes on the full set, we want to construct a subset of Atari-100k with following properties:

1. Since we are interested in *speeding up* the algorithm, we ought to select such environments, in which the method makes consistent progress in a given extended interaction budget. For example, given a maximum speedup target of $15\times$, we would focus on the horizon of $15 \cdot (400 \times 10^3) = 6 \times 10^6$ environment steps. If the method doesn't improve much beyond the performance of a random agent, or solves the task immediately, gauging the speedup factor becomes impossible.
2. The variance of the results with respect to different RNG seeds used should be minimized, so as to be able to derive the expected score with a limited number of RNG seeds with any degree of accuracy.
3. The subset should be as representative of the performance on the whole subset as possible.

First, let us filter out the environments which do not satisfy first two criteria. In the absence of standardized metrics which would inform our choice quantitatively, we resort to visual inspection of the performance curves for each environment in the first 6×10^6 steps. The relevant data can be found in Figure 4.1. Based on it, we select following games for further evaluation: **Amidar**, **Assault**, **Asterix**, **CrazyClimber**, **JamesBond**, **MsPacman** and **Pong**.

Next, we shall select out of this subset 3 games, which are most predictive of the performance on the complete 26-game benchmark. We follow the approach of (Atari-5), and use the source code provided by the authors to arrive at the final result. The three environments selected by the algorithm are: **Assault**, **CrazyClimber** and **MsPacman**.

We use the same environment settings (frame skip values, time limit etc.) as the original DreamerV2. (See Appendix for more details?)

The original implementation of DreamerV2 is written in Tensorflow - we have ported the relevant source code (mainly the implementation of the world model and the actor critic) to Pytorch, resulting in slight changes in overall performance. It remains, however, comparable:

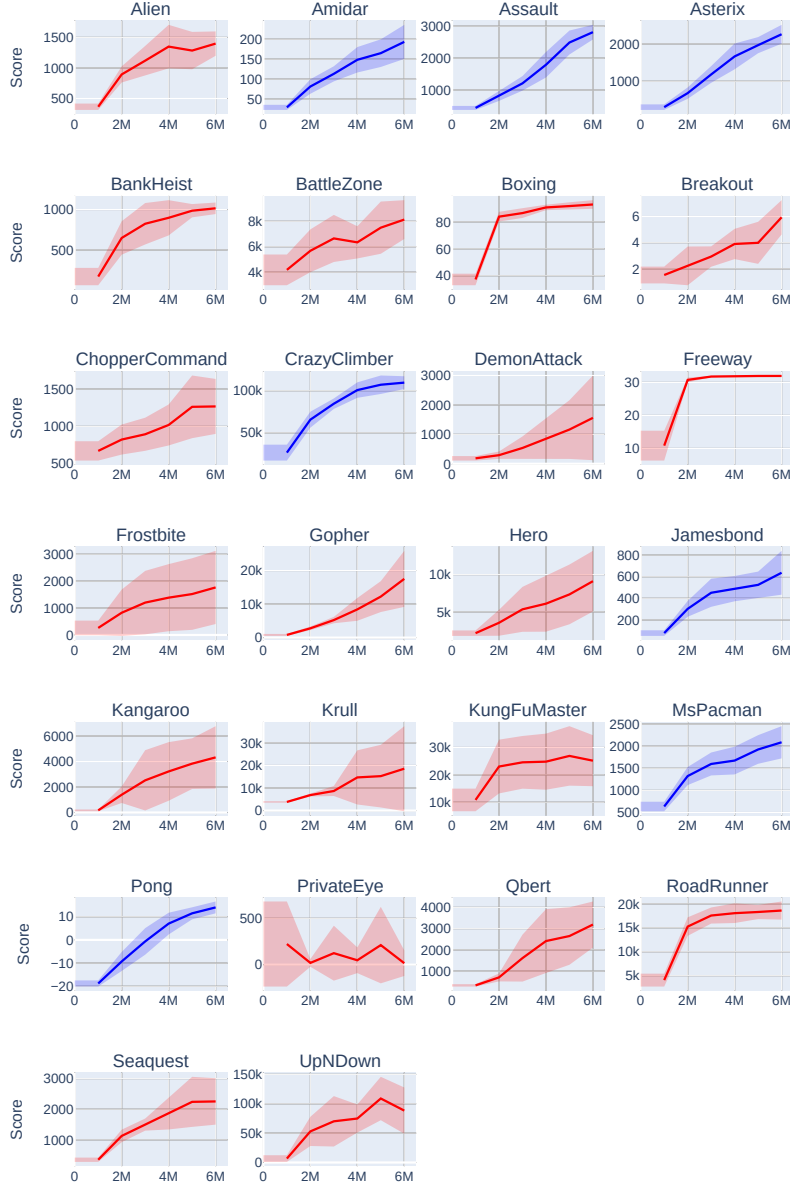


Figure 4.1: Performance curves for Atari-100k, with default settings. The values have been provided by the authors of the DreamerV2 paper. The x-axis represents the number of environment steps passed. The y-axis represents the validation score. Blue curves indicate environments, which we have selected for further evaluation.

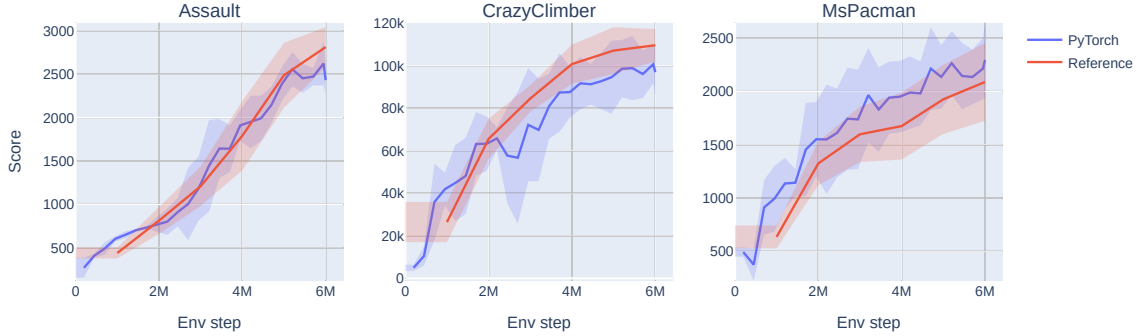


Figure 4.2: Comparison of the validation scores for Tensorflow and Pytorch versions. *Note:* The scores for the reference (Tensorflow) version have been supplied by the authors, and were not verified independently.

Figure 4.2 depicts the validation score curves for both the original implementation, and the Pytorch version.

4.2. Training recipe optimization

As mentioned, many model-free algorithms can be made more efficient through training recipe-level improvements. We wish to extend these findings to the domain of model-based RL. This is, however, non-trivial, and requires further investigation.

First, there are purely technical difficulties. Let us, for example, consider scaling up the frequency of model and RL updates. As opposed to the model-free case, we are dealing with two separate learning processes, that of the model and of the RL module, resulting in two separate update frequencies to be tuned. If we were to search for the optimal configuration through a grid search, we would be faced with a quadratic number of training runs to perform, relative to the model-free case, which would quickly become infeasible. It is therefore imperative to reduce the extent of the hyperparameter sweep as much as feasible.

Next, insofar as the collapse of the performance for regular model-free RL agents, when increasing the update-to-data frequency, is attributed to plasticity loss, it is not immediately clear whether model-based agents undergo the same process, whether the model and the RL module suffer from the primacy bias or perhaps only one of them does, and whether interventions proposed in the literature prove beneficial here as well. If such interventions require tuning, as is for example the case for periodic resets, then we possibly encounter another computationally expensive grid search.

The strategy is as follows:

4.2.1. Base architecture

We choose DreamerV2 architecture as the baseline for the purposes of optimizing the training recipe. The choice is driven by a number of factors:

1. We wish to start with a non-sample-efficient setup, and (hopefully) make it more data-efficient.

2. We want to focus on the visual domains, such as Atari-100k.
3. The model and the RL module are, asymptotically, capable of reaching high performance, meaning we will not be bottlenecked by the insufficiency in the model/RL modules themselves.

The agent consists of two modules:

- The world model – an observation encoder and a Recurrent State Space Model (RSSM) for modeling stochastic latent dynamics.
- The RL module – an actor-critic algorithm, optimized either via backpropagation on the values themselves, or through the use of policy gradients.

The training loop is structured as depicted in Algorithm 1.

Algorithm 1 Dreamer training recipe

- 1: Gather P env samples with an agent acting randomly, and put them into a replay buffer
 - 2: **while** Total number of env steps $< T$ **do**
 - 3: Optimize the world model using batch of real rollouts $(o_{1:L}, a_{1:L-1}, r_{1:L-1}, \gamma)$ drawn from the replay buffer.
 - 4: Optimize the RL agent using the batch of dream data $(s_{1:H}, a_{1:H-1}, \tilde{r}_{1:H-1}, \tilde{\gamma}_{1:H})$, generated by starting from the states obtained through an inference process on real-world rollouts, and imagining next $H - 1$ states with the world model.
 - 5: Perform $E = 64$ environment steps using the behavior policy, and store them in the replay buffer.
 - 6: **end while**
-

For more details, we refer the reader to the source paper. For our purposes, though, we wish to treat these modules as “black boxes”.

4.3. Naïve speedup test

We begin with an experiment involving a straightforward increase of the frequency of optimization steps, without any further changes. The goals are as follows: to establish a baseline for further analysis, to find the update-to-data regime where the agent performance degrades, and to investigate what happens at that point with the networks in question.

Setup Base DreamerV2 training recipe begins with a prefill of $P = 200 \times 10^3$ and continues until the total step count is $T = 200 \times 10^6$. The latter loop consists of $E = 64$ env steps (at the frame skip value of 4, this corresponds to 16 agent steps) and a single optimization step.

We replace T by 400×10^3 to match Atari-100k benchmark environment budget, and P with 20×10^3 . Then, we perform tests with varying values of $E \in \{64, 32, 16, 8, 4, 2\}$, for a set of 3 selected environments and 5 RNG seeds.

Beyond that, for testing purposes, we also add an online train-validation split mechanism: each episode in the replay buffer is assigned either to the training set or the validation set – to be precise, every $(1/p)$ -th episode is assigned to the validation set, where $p = 0.15$.

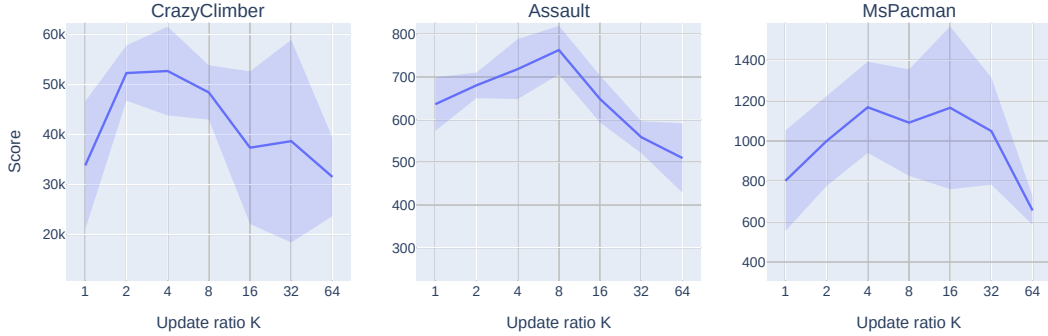


Figure 4.3: Final test episode returns for each of the 3 tested environments, relative to the data-to-update ratio E . The line indicates the mean across the RNG seeds, and the shaded area indicates the standard deviation. *Note*: The x-axis is logarithmic.

Results Let us first analyze final test performance, depending on the data-to-update ratio E , for each environment (Figure 4.3). The mean final score is maximized for a given region of the values of E , beyond which it decreases, presumably either due to under- or overoptimization. For each task, the optimal value is different: 2 – 4 for **CrazyClimber**, ≈ 8 for **Assault** and a wide range for **MsPacman**.

4.3.1. Decoupling model and RL optimization

As noted previously, the optimal training schedules for the model and the RL module need not coincide - we may, for example, find that the model is overfitting, yet the RL head remains underoptimized. The next experiment aims to investigate the empirical effects of decoupling model and RL update frequencies.

Setup We adjust the code, so that the model is optimized once every K_{WM} environment steps, and the RL module is optimized once every K_{RL} environment steps. A grid search is performed for **Assault** environment [Extend to all three, if there’s enough compute time], with ratios $K_{WM}, K_{RL} \in \{2, 4, 8\}$, corresponding to the observed regime, where the performance starts decreasing.

Results Figure 4.4 shows the agent performance depending on the configuration of the ratios. Both the model and the RL ratios play a significant role in determining the final score, though no clear pattern can be determined - for example, increasing RL update ratio does not necessarily improve the performance.

4.4. Investigating the learning dynamics of the agents

So far, the experiments suggest that achieving optimal or near-optimal performance may require, at the very least, a hyperparameter sweep over *both* model and RL update frequencies. Given the increase in the number of training runs this would require, we will now investigate methods to auto-select at least some of these parameters. In order to achieve this, we will

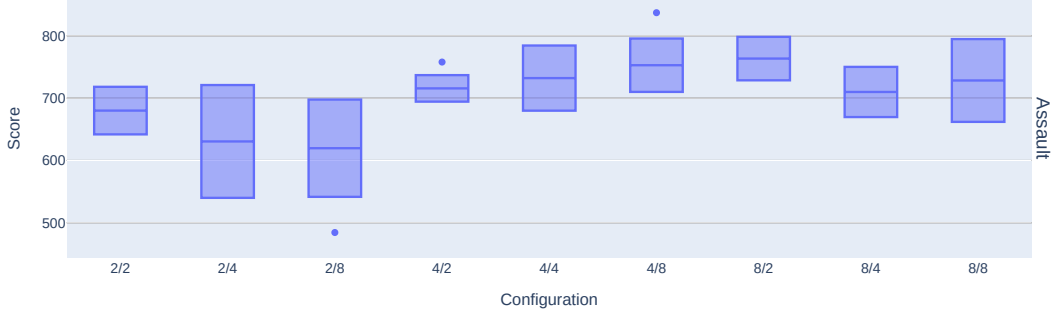


Figure 4.4: Final validation score for each model/RL update ratio configuration. The x-axis represents each run, marked by a tag in the form of ' K_{WM}/K_{RL} '. The points represent the scores for each seed, with the corresponding box plot to the right.

start with an examination of the various metrics throughout the run, and see whether they are correlated with the final score. The idea is that if we are able to control them throughout the optimization process, we may be able to guide them in such a way so as to achieve higher final performance.

4.4.1. Analyzing the world model

Since model learning is a supervised learning task, the most important metric to monitor is, we might presume, the validation loss. We compute it as follows: throughout the training process, we will load a batch of data using the validation episodes, and compute the mean model loss. We will start with the aggregate results - the relationship between the final validation loss and the final agent performance, for the equal-ratio experiment described in section 4.3. As we can see in Figure 4.5, it would appear, somewhat surprisingly, that the value, for individual trials, is not meaningfully correlated with the final score achieved by the agent. A closer inspection of the validation loss curves (Figure 4.6) reveals, that the agents do not, in general, exhibit classical overfitting behavior, in the sense that the validation loss curves plateau at worst, and in general the loss value keeps decreasing. The learning process nevertheless degrades, as the final values of the validation loss keep increasing as the ratio of the optimization and environment steps increases. The ratio values, for which the final loss value is the smallest, do not correspond to the ones which achieve the highest performance.

To further examine the behavior of the agent throughout the optimization process, let us take a look at the *training* loss curves. An aggregate comparison (Figure 4.7) reveals a degree of correlation between the agent performance scores and the training loss values. The loss curves (Figure 4.8) are of altogether different nature, compared to the validation loss curves. We can see a clear monotonic relationship between the final loss value achieved and the update frequency, though, as before, the better values do not lead to improvements in terms of the quality of the agents. Perhaps more interestingly, at very high update frequencies (i.e. small data-to-update ratios), the optimization process seems to stall out completely. For the value of $K = 1$, for example, in all three environments tested, near-“optimal” value is reached after the first 100×10^3 environment steps, whereafter it either improves marginally, or even increases.

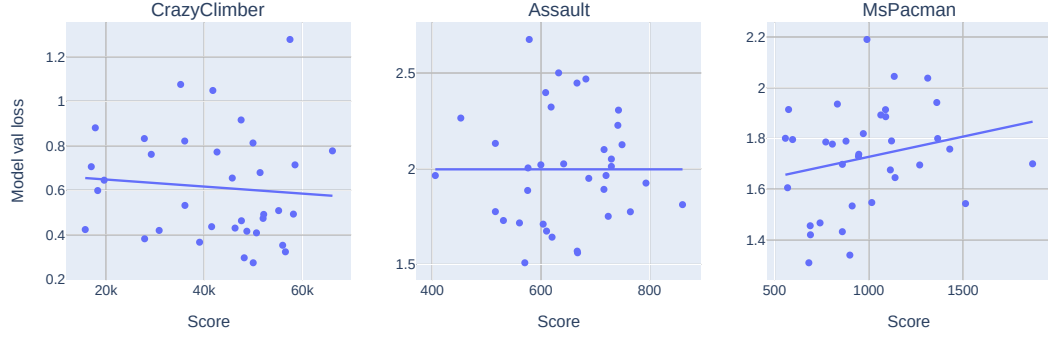


Figure 4.5: Relationship between final validation score and final model validation loss. Each point represents a single training run. The x-axis represents the agent performance. The y-axis represents the value of the model validation loss at the end of the training.

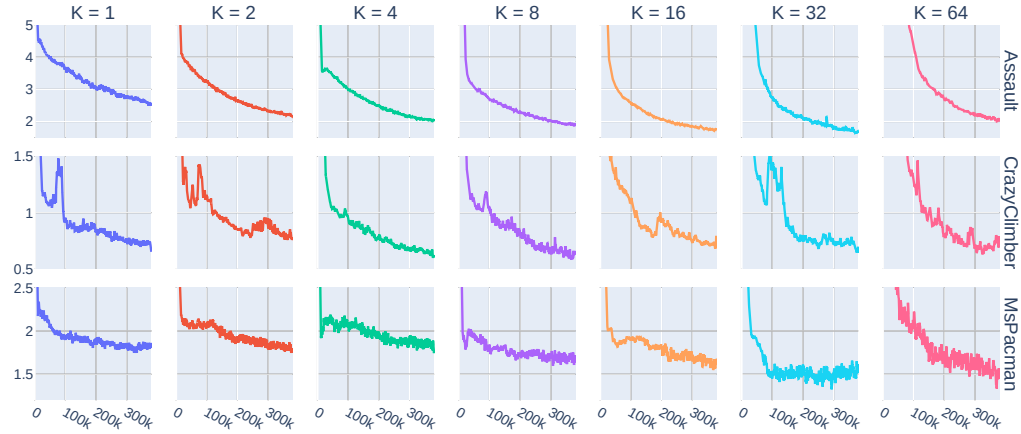


Figure 4.6: Averaged model validation loss curves for each configuration of the environment and the update ratio used. Rows represent different environments, and the columns the different update ratios used. The x-axis corresponds to the number of environment steps taken, and the y-axes represent loss values. The ranges of the y-axes are different for each environment.

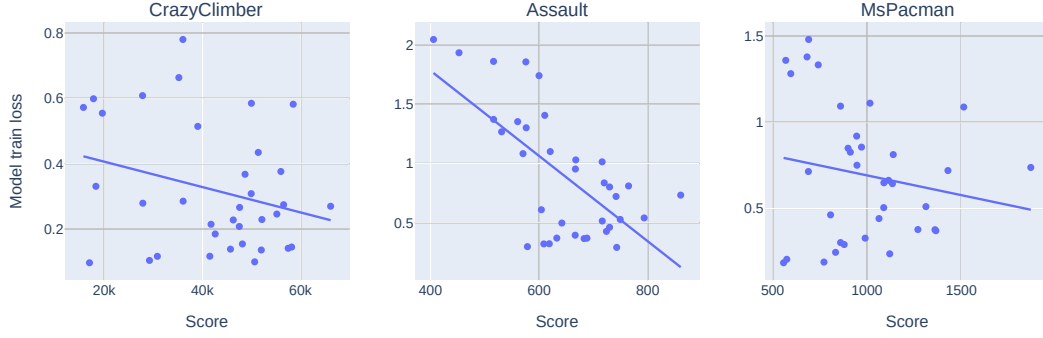


Figure 4.7: Relationship between final validation score and final model training loss. Each point represents a single training run. The x-axis represents the agent performance. The y-axis represents the value of the model training loss at the end of the training.

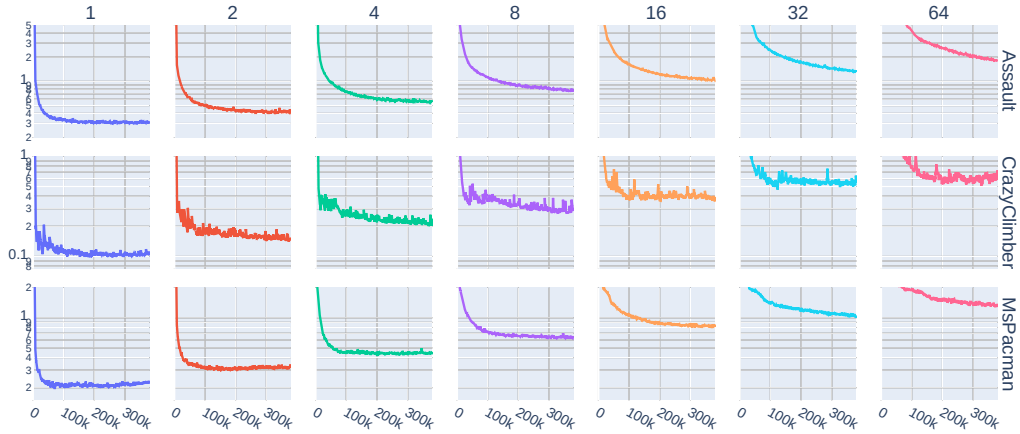


Figure 4.8: Averaged model training loss curves for each configuration of the environment and the update ratio used. Rows represent different environments, and the columns the different update ratios used. The x-axis corresponds to the number of environment steps taken, and the y-axes represent loss values. The ranges of the y-axes are different for each environment, and are **logarithmic** due to the wide range of values.

4.4.2. Plasticity analysis

As we have noted previously, a number of works have identified the phenomenon of plasticity loss, meaning the inability of neural networks to learn from more recent data samples in the continual learning setting, as a possible culprit in the degradation of the RL agent performance. It might prove fruitful to investigate, whether something similar happens in the case of model-based RL, and specifically the agents based on Dreamer algorithm.

In the first order, we would like to perform a “post hoc” test of plasticity loss - the results will not allow us to monitor the occurrence of the phenomenon *during* training, but will show whether the degradation has occurred. We follow the strategy of [1], wherein an agent is trained, starting from the neural network parameters of a previously trained agent. In that study, the process is repeated 10 times. Due to computational constraints, we only perform a single repetition. Also, the environments under investigation there were drawn from the Coinrun suite of randomized environment. As we are interested in Atari tasks, and there is no standard way to randomize the environment - the idea of changing the Atari game does not work due to different action spaces, meaning different agent architectures required - we devise a custom randomization scheme for Atari. To be specific, at the start of the training run, we choose (1) a particular permutation σ of actions, meaning an action $a \in \{0, \dots, A-1\} = \mathcal{A}$ is mapped to $\sigma(a) \in \mathcal{A}$; (2) whether to perform horizontal flip of the video frames; (3) whether to perform a vertical flip of the video frames. These randomizations have been selected in order to ensure, that the warm-started agent approximately achieves the performance of a random agent, and that the observation and action statistics, e.g. the mean and standard deviation of the pixel values, remain the same.

The training recipe follows the reference test with the default DreamerV2 settings and an environment step budget of 6×10^6 , used for comparing the Pytorch and Tensorflow implementations. The choice is motivated by the smaller variance of the results compared to the time horizon of 400×10^3 used for the other experiments. We load a checkpoint from a baseline test, with the update ratio value of $K = 1$, representing the checkpoints most affected by the primary bias. The environment used during the training run corresponding to the checkpoint is the same, but we change the RNG seeds to introduce further randomization.

(Results)

4.4.3. Analyzing the RL component

Analysis of the behavior of the RL component is less straightforward, as there is no analogue of under- or overfitting of supervised models in the case of reinforcement learning. There are, however, a couple of phenomena, which can cause performance collapse, such as:

1. Value overestimation: Since most RL algorithms compute value estimates via bootstrapping, it is possible for bad initial value estimates to be amplified by the optimization process.
2. Model exploitation: In the context of model-based RL, and specifically the methods based on Dreamer architecture, since all the data given to the RL module is imagined by the model, inaccuracies thereof can cause the agent to learn wrong policies which perform well in the model-induced MDP.
3. Policy collapse: for actor-critic methods, without any regularization, the policy will tend to rapidly concentrate on the actions with the highest advantage estimates, which in turn would hurt exploration and prevent the agent from learning more optimal behaviors.

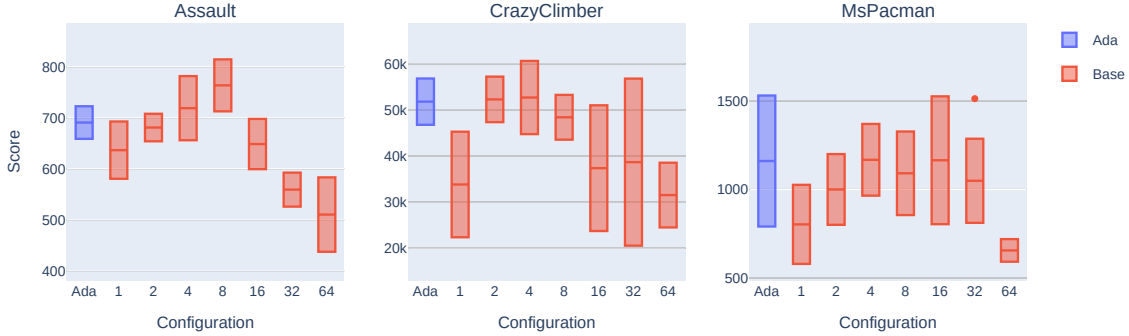


Figure 4.9: The comparison of the final validation scores, for the adaptive ratio experiment *Ada* and the baseline experiments 1 - 64, where the number denotes the model/RL update ratio K . Each subplot shows the results for a different environment tested. The bars signify mean and one standard deviation, obtained from a sample of 5 training runs for each configuration.

To investigate these, we will first take a look at the value curves. [?]

4.5. Auto-tuning update frequencies

It would therefore appear that these metrics cannot be directly used for the purposes of guiding the optimization process. There exist, however, works doing precisely that, which demands a further inquiry.

We will specifically do a closer examination of (Dynamic update ratios paper). The approach taken by the authors is to check validation loss throughout the training process, and to decrease the update frequency if the loss value has degraded, and increase it if it has improved. To start off, we will replicate the experiment, albeit with some notable changes:

1. The authors propose using image reconstruction loss as the validation metric. We would rather have the method apply to models other than DreamerV2's specifically, however, so will use the total loss value.
2. A validation set is constructed differently: 3×10^3 transitions are collected every 100×10^3 steps, which corresponds to the flat 12% of the transitions being reserved for the validation set, whereas we reserve 15% of the *episodes* instead. Thus, the training set contains transitions which immediately precede or succeed the ones in the validation set - whether it represents a form of data contamination is unclear.

The results can be found in Figure 4.9, along with the baseline results for the sake of comparison. It would appear that the strategy is, in general, effective, though not universal - for example, the scores on **Assault** do not quite reach the performance peak.

Given that the idea behind this approach is to minimize model overfitting by monitoring validation loss, our next step is to look at the validation loss. The results (Figure 4.10) do not seem to imply that the method has a consistent positive effect on the model validation loss. In fact, if we compare validation loss curves and the current ratio values (Figure 4.11), the relationship is not precisely what we would expect. As noted previously, the idea behind the

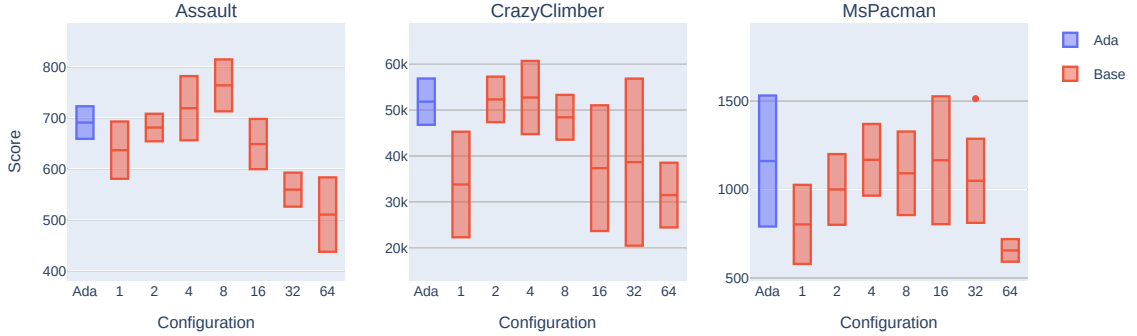


Figure 4.10: The comparison of the final model validation loss values, for the adaptive ratio experiment **Ada** and the baseline experiments 1 - 64, where the number denotes the model/RL update ratio K . Each subplot shows the results for a different environment tested. The bars signify mean and one standard deviation, obtained from a sample of 5 training runs for each configuration.

method is to increase update ratio as long as the validation loss decreases, and vice versa. In short time scales, specifically the 2×10^3 environment steps between each ratio update, this does occur, but across longer time spans, the coupling breaks. For example, for **MsPacman** and the RNG seed of 1, the validation loss decreases up until the timestep of 200×10^3 , yet the ratio keeps increasing. After the jump, presumably due to the influence of a new validation episode in the buffer, the loss value starts decreasing, and the ratio decreases.

4.5.1. Decoupled adaptive ratio system

It appears, then, that the theoretical underpinnings behind the effectiveness of this approach are not quite as clear as one would hope. Despite this, we believe one can nevertheless improve upon this method in a principled fashion. Drawing on the results concerning the decoupling of the optimization rates for the model and the RL modules, we propose a similar decoupling in the adaptive regime. In particular, we will make only the model update ratio adaptive, and keep the RL update ratio fixed at a pre-defined value. The results (Figure 4.12) indicate, that the decoupling has a positive impact on the agent performance.

4.6. Data augmentation

Incorporating data augmentation into an RL framework can be done in non-trivial ways. DrQ, for example, combines transforming the input, averaging the Q functions across multiple augmented samples, and averaging the Q targets. In the context of model-based RL, one could develop similar schemes for both the world model and the RL modules, e.g. by averaging posterior state distributions over multiple image augmentations. This, however, would require changing the implementation of these modules, whereas our goal is to make the framework as modular as possible. Therefore, we opt to only augment the input, and forego the averaging of various quantities in question.

We follow the original paper in choosing random shifts as the image transformation to

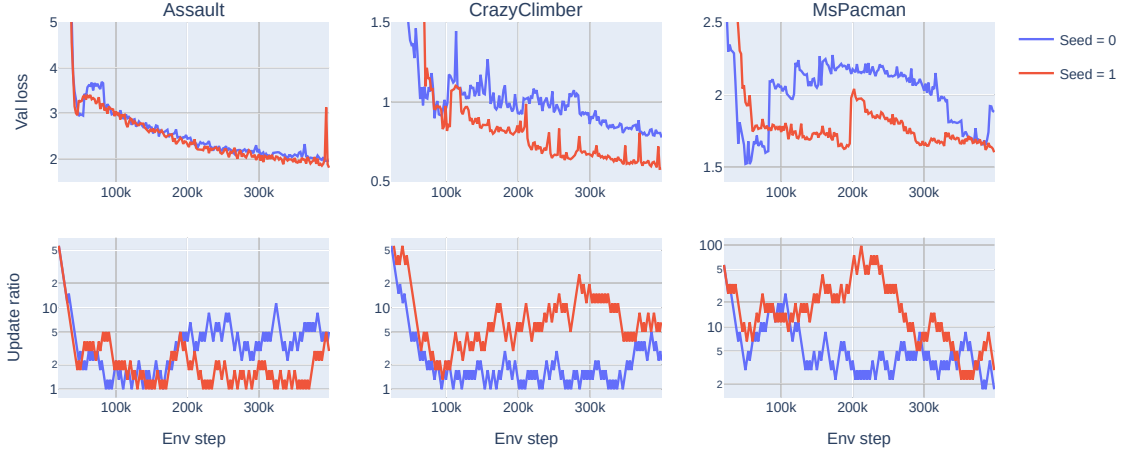


Figure 4.11: The curves for the model validation loss and the current update ratio, for the adaptive ratio experiment. Each column represents a different Atari task. Two training runs, corresponding to two different RNG seeds, are shown on each subplot. The model validation curves have been smoothed with exponential moving average to improve the clarity of the presentation.

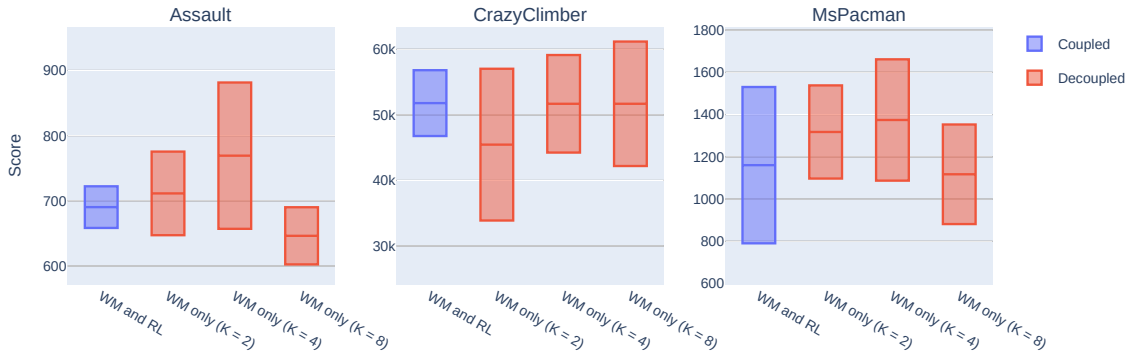


Figure 4.12: A comparison between the final validation scores for the coupled and the decoupled variants of the adaptive update ratio experiment. Each subplot represents a different environment tested. Each box signifies the results for a given configuration - mean and one standard deviation, obtained from a sample of 5 training runs for each configuration.

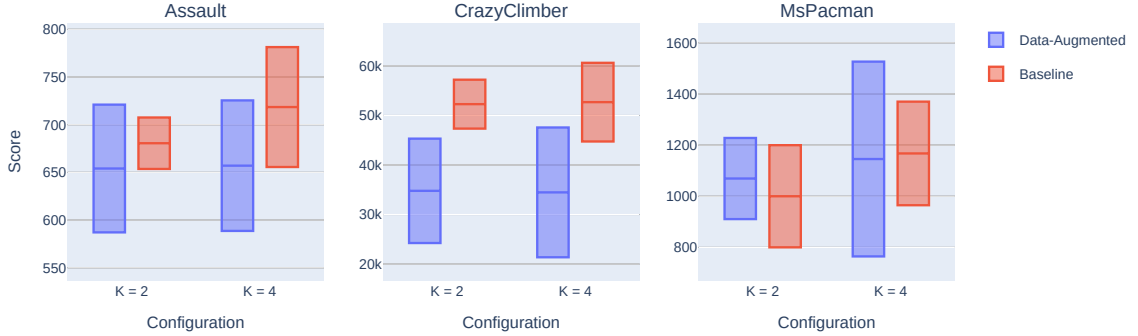


Figure 4.13: A comparison of the final validation scores for the data-augmented agent, and a non-data-augmented baseline. Each subplot represents the results for a single environment. In each subplot, results for different values of model/RL update ratio K are grouped together. Each group contains a box, signifying the mean and the standard deviation evaluated across 5 training runs, for the data-augmented and the non-data-augmented variants.

apply. The augmentations will be applied to both images in the batch for the model optimization, and the batch used to generate the initial states for the imagined rollouts. As far as the training recipe is concerned, we will start with a fixed-ratio setup in order to compare the data-augmented recipe with a non-data-augmented baseline.

Results The comparison of the test episode returns for the two variants (Figure 4.13) shows, that, with the base settings, the addition of data augmentation *degrades* performance, excepting perhaps **MsPacman**. As the main idea behind data augmentation is the reduction of model overfitting, we will next investigate the training and the validation loss curves for the model. The analysis of the final validation loss for each variant (Figure 4.14) reveals, that not only is it significantly higher compared to the baseline variant, but it also does not improve as the update frequency increases (equivalently, K decreases).

If we take a look at the model training loss curves (Figure 4.15), we may observe a following phenomenon: for **Assault** and **CrazyClimber**, the addition of random shifts appears to slow down, or even stall out the training process, as shown by the slower rate of decrease (in log-space). For **MsPacman**, however, the reverse is true: in the original experiment, training loss stops improving at a certain point, whereas in the data-augmented scenario, we observe a consistent decrease. [What further conclusions to draw?]

As suggested in (DrQ paper citation), the type of image augmentation used has a large impact on the effectiveness of the method. To this end, the next experiment will assess the impact of changing the kind of the augmentation. We mostly follow the source paper in the selection of the transforms to test:

1. **none**: Baseline/No augmentations applied.
2. **shift4**: (Default) Apply a random image shift. The shift in each axis is selected randomly from $\{-4, \dots, 4\}$.
3. **shift2**: Apply a random image shift. The shift in each axis is selected randomly from $\{-2, \dots, 2\}$. This variant of **shift4** is motivated by the fact, that DrQ image size is

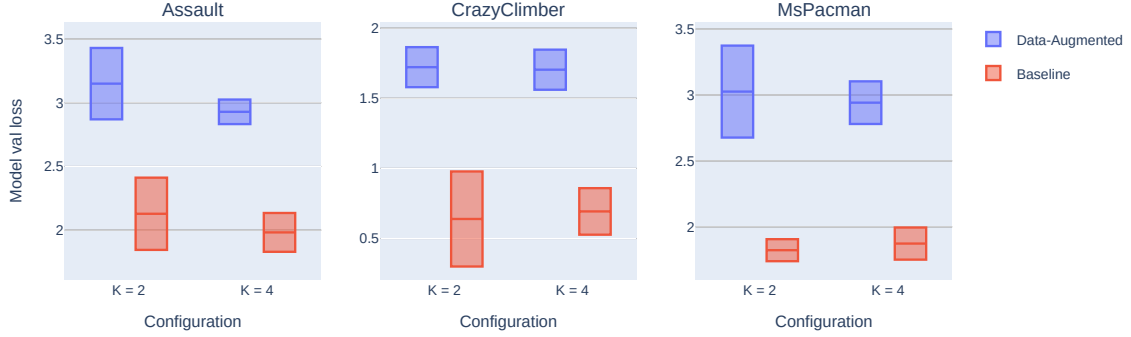


Figure 4.14: A comparison of the final model validation loss values for the data-augmented agent, and a non-data-augmented baseline. Each subplot represents the results for a single environment. In each subplot, results for different values of model/RL update ratio K are grouped together. Each group contains a box, signifying the mean and the standard deviation evaluated across 5 training runs, for the data-augmented and the non-data-augmented variants.

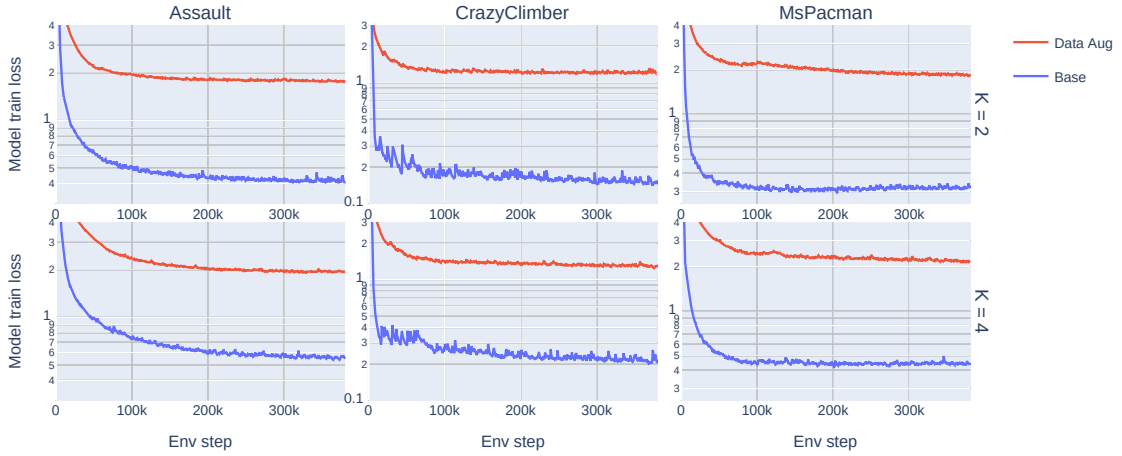


Figure 4.15: A comparison of the model training loss curves for the data-augmented agent, and a non-data-augmented baseline. Each column represents the results for a single environment. Each row corresponds to a different configuration - model/RL update ratio used. In each subplot, curves for the data-augmented and non-data-augmented variants are displayed. The curves have been smoothed by exponential moving average, to improve the clarity of the presentation.



Figure 4.16: The final validation scores for different data augmentation types. Each row contains the results for a different environment tested. Each of the boxes corresponds to a given augmentation type, and displays the mean and the standard deviation measured with a sample of 5 training runs.

84×84 , whereas we operate on images of size 64×64 - it could be possible, that the shift of 4 is too large.

4. **cutout**: Perform random erasing with probability $p = 0.5$.
5. **intensity**: Perform random brightness/intensity increase. To be specific, each pixel's value is multiplied by $f \sim \mathcal{U}[0.95, 1.05]$.
6. **rotate**: Rotate the image by $d \sim \mathcal{U}[-5, 5]$ degrees.
7. **vflip**: Flip the image vertically, with probability $p = 0.1$.

where $x \sim \mathcal{U}[a, b]$ denotes a sample from the uniform distribution over $[a, b] \subset \mathbb{R}$.

The results (Figure 4.16) suggest, that no type of image augmentation has a consistent positive effect on the agent performance.

4.7. Choice of the RL algorithm choice

So far, all the experiments have used the default RL module of DreamerV2, namely a variant of Advantage Actor-Critic (A2C). In this section, we shall investigate the effects of replacing it, whilst keeping the model component intact. Beyond hopefully achieving improvement in terms of agent performance, our objective is to elucidate the design choices when incorporating model-free RL algorithms into a model-based setting.

The RL algorithms we wish to test are Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). The selection is motivated the following factors:

1. Since the original RL algorithm in DreamerV2 uses, for Atari environments, vanilla policy gradients with Generalized Advantage Estimation, the most straightforward way to improve upon it is to replace it with an improved policy-gradient-based algorithm. We select PPO due to high performance and the ability to reuse simulated data, possibly increasing the overall computational performance.
2. Beyond that, we would like to investigate the behavior of Q -learning algorithms in a model-based setting. We select SAC as a strong baseline.

4.7.1. PPO Experiments

We will start with the experiments involving PPO. For the choice of the hyperparameters, we take the approach of replicating the default ones for A2C, whenever applicable. This includes the network architectures of the encoders for the actor and the critic functions, the value of the discount factor γ and GAE discount λ , actor and critic optimizer learning rates, and the entropy penalty α .

There remain two hyperparameters to select: the number of update epochs N , meaning the amount of times each data batch item is used, and the number of minibatches for each update epoch M , being the number of splits of the data batch into minibatches used for each optimization step. Thus, the total number of parameter updates is NM . We may note that, for $N = M = 1$, the procedure is in fact equivalent to the original A2C algorithm.

Beyond that, we need to specify the training recipe. We leave the model update ratio fixed at $K_{WM} = 4$, and finetune RL update ratio K_{RL} , number of update epochs N and number of minibatches M . As the baseline, we select a scenario with $K_{WM} = K_{RL} = 4$. We will examine following configurations:

1. 8/4/1 ($K_{RL} = 8, N = 4, M = 1$) - We exchange less frequent RL optimization steps for a greater number of update epochs within each optimization step.
2. 8/8/1 ($K_{RL} = 8, N = 8, M = 1$) - A variant of the first configuration, with more update epochs.
3. 16/8/1 ($K_{RL} = 16, N = 8, M = 1$) - A more extreme version of the first preset, with an equal number of RL optimization steps, and greater data reuse.
4. 8/4/8 ($K_{RL} = 8, N = 4, M = 8$) - We split the batch into 8 minibatches, increasing the total number of parameter updates by $8\times$.

Results If we take a look at the final validation scores (Figure 4.17), we may infer that using PPO, in any of the tested configurations, has, on average, a positive effect on the performance. The relationship is not consistent, though - for example, on *MsPacman*, the

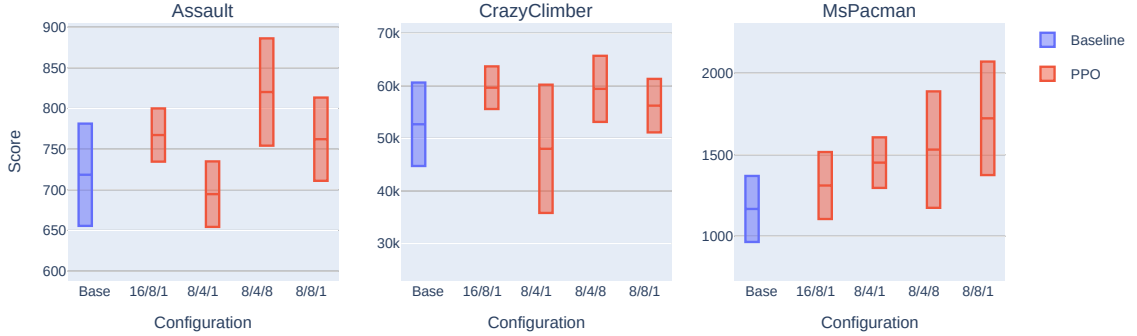


Figure 4.17: Comparison of the final validation scores between the baseline and a number of different configurations of PPO. Each subplot contains the results for a different Atari environment. Each box represents the mean and the standard deviation computed across 5 training runs.

configuration 16/8/1 underperforms 8/4/1, whereas the opposite is the case on the other two environments.

In order for the comparison to be fair, we ought to make sure, that the total training time does not vastly exceed the baseline. As Figure 4.18 indicates, except for the preset with $M = 8$, the training time is either the same, or even lower, even though the number of RL optimization steps is higher. This is likely due to the smaller number of imaginary batches, that need to be constructed.

4.7.2. SAC Experiments

Let us now conduct the experiments with SAC algorithm. In the first order, let us discuss the extent of the hyperparameter sweep to conduct. A complete grid search over network architectures, optimizer parameters etc. is clearly infeasible. We take the approach of reusing A2C’s actor architecture and optimizer, and using A2C’s encoder and optimizer settings for the value function as a proxy for SAC’s Q functions. As both algorithms use the target functions, we copy the target Q function update schedule from A2C - a full copy every 100 optimization steps, to be precise.

The most impactful hyperparameter left, then, is the choice of temperature/reward scale α . As the first step, we perform a random search over the value of the parameter. Ideally, we would perform a grid search, testing each value of α on the set of three environments and five RNG seeds. However, such an experiment would be computationally onerous. What we do instead, rather, is the following: for each pair of the environment/task and the RNG seed, we select the value of α from a log-normal distribution: $\log \alpha \sim \mathcal{N}(\log 10^{-3}, \log 10)$. In other words, we expect the first standard deviation to cover the interval $[10^{-4}, 10^{-2}]$. To increase the coverage, multiple α values can be tested for each pair. Then, we shall estimate the optimal value of α .

Results Let us start with the analysis of the final scores (Figure 4.19) Interestingly, the choice of α regularizer does *not* seem to significantly impact the performance. If we take a look at the final entropy values (Figure 4.20), which ought to be the primary metric controlled

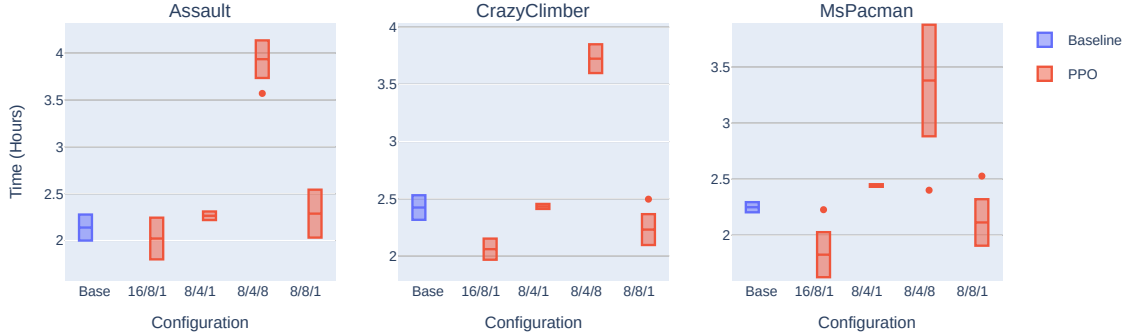


Figure 4.18: Comparison of the total training run durations for the baseline and a number of different configurations of PPO. Each subplot contains the results for a different Atari environment. Each box represents the mean and the standard deviation computed across 5 training runs.

by the value of α , we may deduce that, while the value does impact the entropy (the effect can be seen most strongly on the chart concerning **Assault**), at a certain point we hit a lower bound of sorts, whereafter further decreasing the value of α does not affect it quite as strongly. The aforementioned lower entropy bound seems to be approximately 10^{-2} for all the environments.

Another method, which can be used to derive the value of α temperature, is auto-tuning, as described in (SAC Alg. and App. citation). Briefly, the idea is to specify a schedule of the mean policy entropy, rather than the α parameter itself, and optimize the value of α so that the schedule is achieved. The optimization is achieved via minimization of loss function $L = \alpha(H - H_0)$, where H_0 denotes the current entropy target.

In order to use it, we must specify the target entropy schedule. The recommendations for the model-free case suggest the use of a constant entropy target of $0.89H_{\max}$ for the discrete action spaces, where $H_{\max}$ denotes the maximum possible entropy for a policy in a given action space. However, the entropy curves of the best-performing agents in the baseline scenario (Figure 4.21) imply, that we ought to specify a rapidly decaying entropy schedule, with the final mean entropy value of approximately $0.05H_{\max}$.

With this in mind, we will now proceed with an experiment, wherein we will set the entropy target to follow the dotted line on Figure 4.21 - to be precise, we start at $0.99H_{\max}$, descend to $0.1H_{\max}$ by the environment step 50×10^3 , and reach the target of $0.05H_{\max}$ by the environment step 200×10^3 . As for the optimizer for the α loss value, we choose Adam with learning rate 2.5×10^{-3} , $\epsilon = 10^{-5}$ and leave other parameters as default in Pytorch.

The results (Figure 4.22) are inconclusive - performance on **MsPacman** is significantly improved, remains approximately the same on **Assault**, and is degraded on **CrazyClimber**.

4.8. Combining the improvements

Our experiments have singled out two major avenues for improvement, insofar as the agent performance is concerned: the adaptive update ratio schema, and the use of PPO instead of the default A2C. We have, however, tested them in isolation. It remains for us to see what

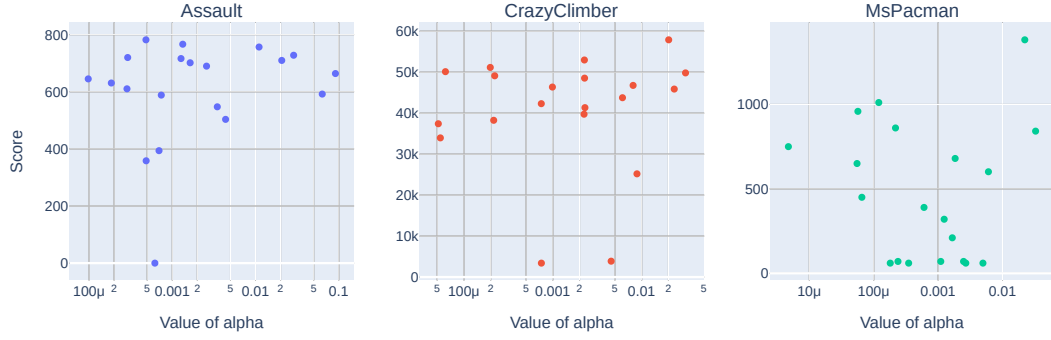


Figure 4.19: Relationship between SAC's α value, and the agent performance. Each subplot contains the results for a different Atari environment. Each point represents a single training run. The x -axis denotes the value of α parameter used in the test. The y -axis signifies the final validation score obtained by the agent.

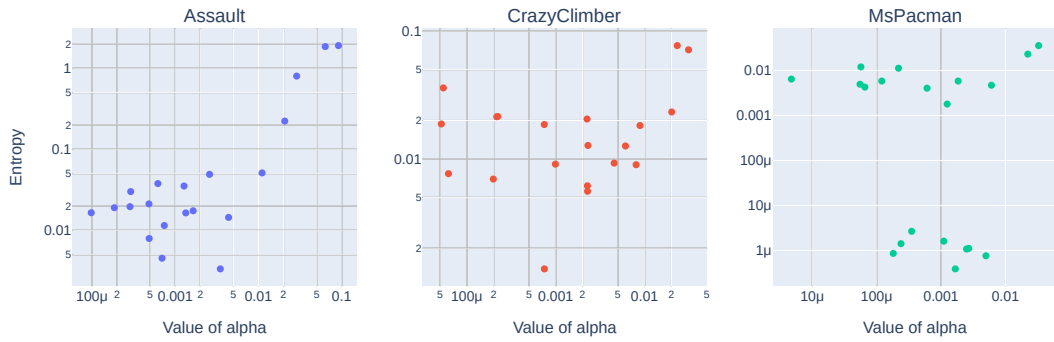


Figure 4.20: Relationship between SAC's α value, and the final mean policy entropy. Each subplot contains the results for a different Atari environment. Each point represents a single training run. The x -axis denotes the value of α parameter used in the test. The y -axis signifies the final value of the mean policy entropy metric.

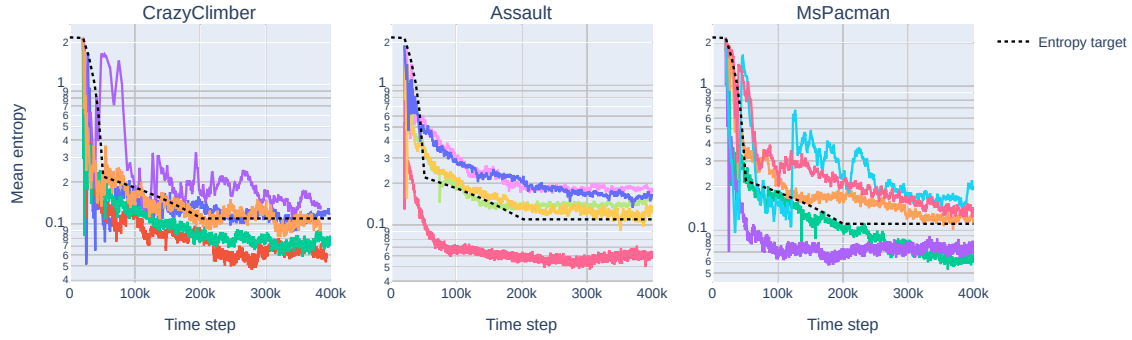


Figure 4.21: Mean policy entropy curves for the best-performing agents in the baseline experiment. Each Atari environment is represented by a different subplot. The x -axis denotes the number of environment step passed. The y -axis represents the mean policy entropy. Each solid line represents a different training run. The dashed line denotes an example entropy schedule approximating these schedules.

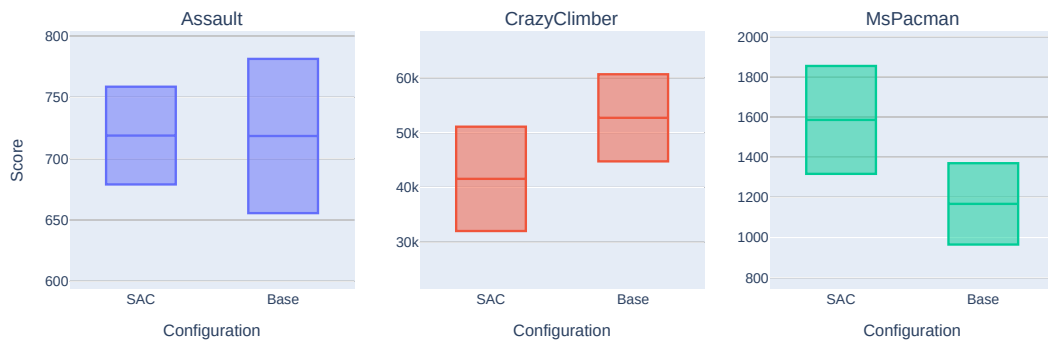


Figure 4.22: A comparison between the baseline experiment and the experiment with SAC and a user-defined entropy schedule. Each Atari environment tested is represented by a single subplot. The boxes denote the mean and the standard deviation of the final validation score, estimated with 5 training runs per configuration.

happens if we employ them all at once. We shall also see the effect of these ablations on the 23 unseen environments.

Due to the requisite computational resources required to run the entire benchmark (26 games $\times$ 5 RNG seeds per single configuration), we will first examine the effects on the three test environments, select the hyperparameters to be used, and then run the benchmark on the chosen set.

4.8.1. Hyperparameter selection

In reference to the adaptive ratio system, the results suggest the use of a decoupled setup, wherein the RL update ratio is fixed. The performance gain is, however, dependent on the specific update ratio used. The validation scores obtained in the PPO experiment, on the other hand, indicate the use of either 16/8/1 or 8/8/1 configuration, the configuration name $K_{\text{RL}}/N/M$ denoting the RL optimization ratio, number of PPO epochs and number of PPO minibatches respectively. We exclude the experiments with $M > 1$ due to excessive adverse effect on the training speed, despite possibly yielding improved results.

These findings suggest a following setup: we will set the RL update ratio fixed, and focus on the hyperparameters for the PPO. We will test a slightly wider range of presets: 8/8/1, 16/8/1, 4/8/1 and 8/4/1.

Chapter 5

Discussion

Bibliography

- [1] Arthur Juliani and Jordan T. Ash. *A Study of Plasticity Loss in On-Policy Deep Reinforcement Learning*. arXiv:2405.19153 [cs]. May 2024. DOI: 10.48550/arXiv.2405.19153. URL: <http://arxiv.org/abs/2405.19153> (visited on 08/29/2024).
- [2] Lukasz Kaiser et al. *Model-Based Reinforcement Learning for Atari*. arXiv:1903.00374 [cs, stat] version: 5. Apr. 2024. DOI: 10.48550/arXiv.1903.00374. URL: <http://arxiv.org/abs/1903.00374> (visited on 08/18/2024).